

OBJECT ORIENTED PROGRAMMING

UNIT - I

**AI303PC: OBJECT ORIENTED PROGRAMMING
USING JAVA**



NEED FOR OOP PARADIGM

- OOP is an approach to program organization and development, which attempts to eliminate some of the drawbacks of conventional programming methods by incorporating the best of structured programming features with several new concepts.
- OOP allows us to decompose a problem into number of entities called objects and then build data and methods (functions) around these entities.
- The data of an object can be accessed only by the methods associated with the object.

OOP PARADIGMS

- Some of the Object-Oriented Paradigm are:
- Emphasis is on data rather than procedure.
- Programs are divided into objects.
- Data Structures are designed such that they
 - Characterize the objects.
- Methods that operate on the data of an object are tied together in the data structure.
- Data is hidden and can not be accessed by external functions.
- Objects may communicate with each other through methods.

SUMMARY OF OOPS

- The following are the basic oops concepts: They are as follows:
- Objects.
- Classes.
- Data Abstraction.
- Data Encapsulation.
- Inheritance.
- Polymorphism.
- Dynamic Binding.
- Message Passing.

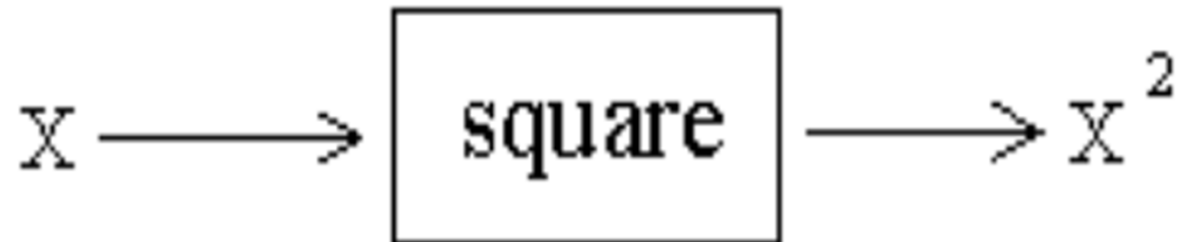
COPING WITH COMPLEXITY

- **"Coping with complexity"** in OOP is all about breaking down large problems into smaller, manageable objects using *abstraction, encapsulation, inheritance, and polymorphism*. It makes code easier to read, reuse, debug, and maintain.
- **Design Patterns & Principles:**
 - Using standard solutions like **MVC**, **Singleton**, or **Factory Pattern** helps structure complex code effectively.
 - Following **SOLID** principles (Single Responsibility, Open/Closed, etc.) makes systems easier to understand and evolve.

ABSTRACTION IN OOP

(PROCEDURAL ABSTRACTION)

- Procedural Abstractions organize instructions.
- Let's first address the problem of squaring a number. Instead of having to write the expression $r * r$ every time, it would be nice to have a black box, or a procedure, that would take in a numerical argument and return the square of that argument.
- In Java, we can create a procedure called square as follows:
- **`double square(double x) { return (x * x); }`**
- If we had this, we could write
- **`square(837)`**



ABSTRACTION IN OOP (DATA ABSTRACTION)

- Data Abstractions organize data.
- **Data types.** A data type is a set of values and a set of operations on those values.
- **Abstract data types (ADT).** An abstract data type is a data type whose internal representation is hidden from the client
- **Example:** Java's *String* data type is an important and useful ADT.

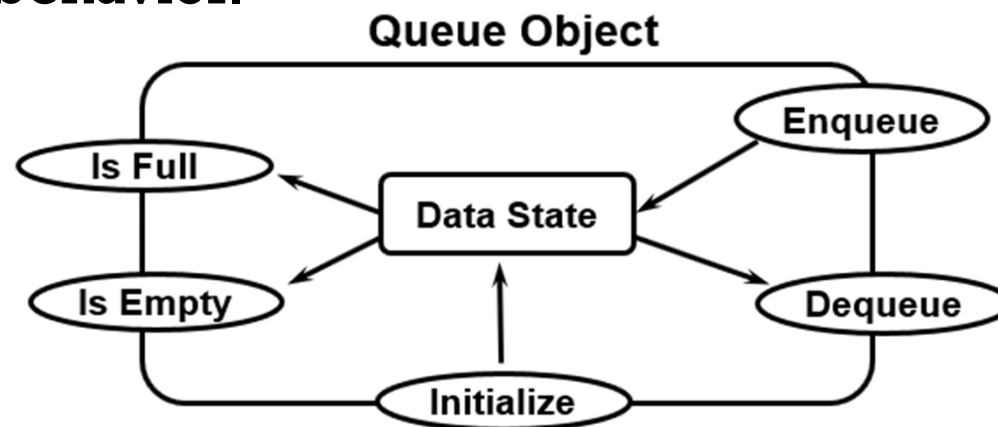
StudentType

Name (string)
Marks (num)
Grade (char)
Student Number (num)

ABSTRACTION IN OOP

(BEHAVIORAL ABSTRACTION)

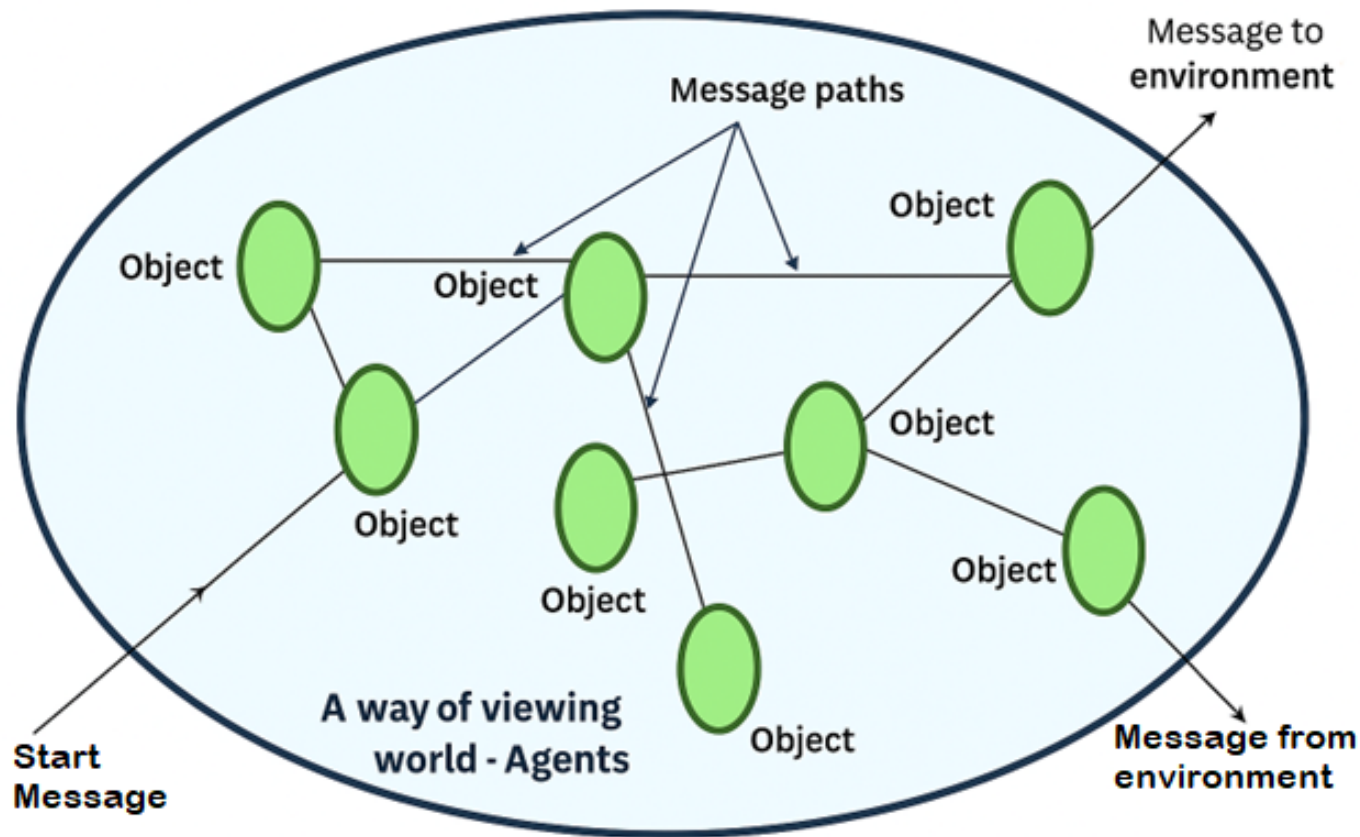
- Behavioral Abstractions combine procedural and data abstractions.
- Behavioral abstraction, also known as **control abstraction**, focuses on hiding the complex implementation details of how an operation or method is performed. It defines a contract for behavior without specifying the exact steps involved in achieving that behavior.



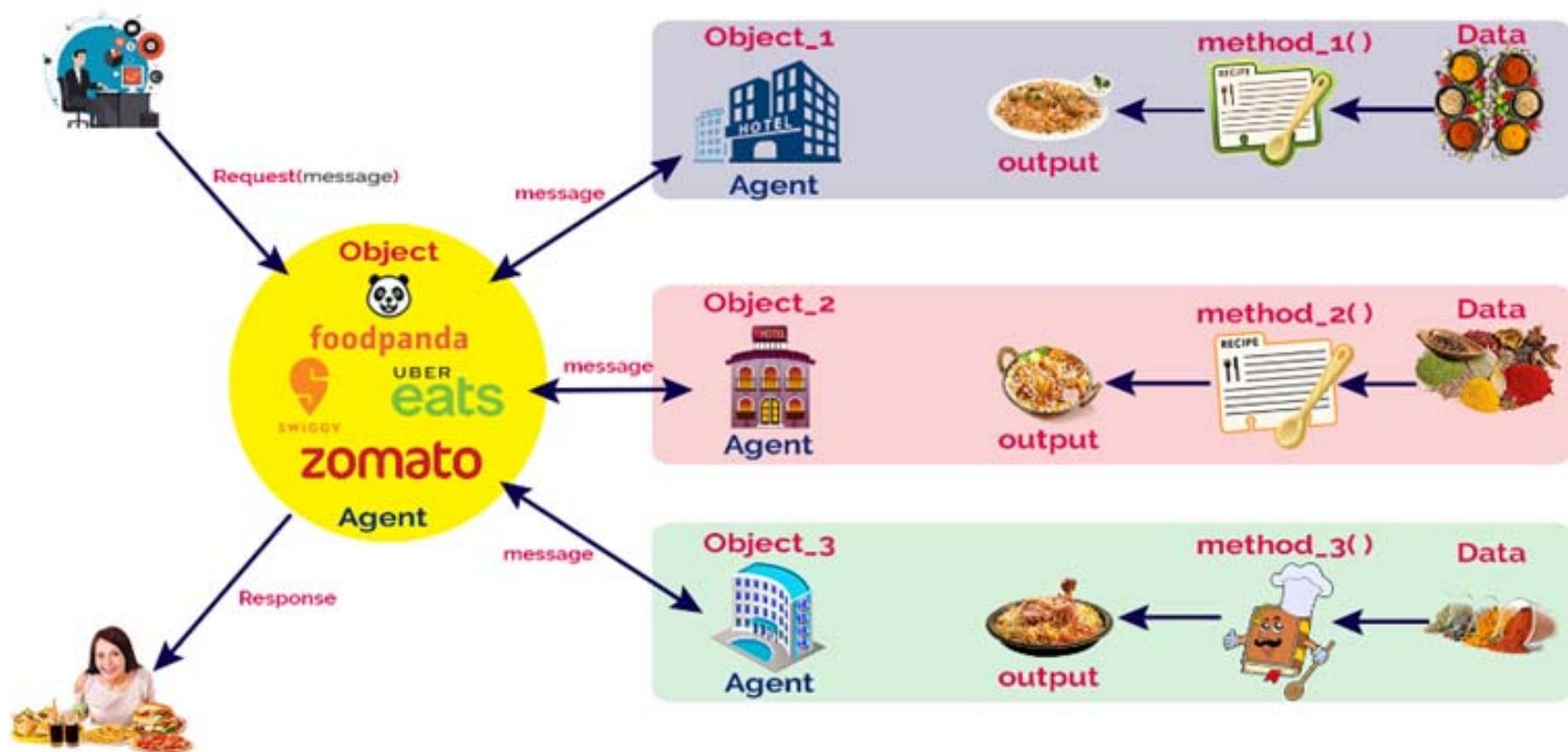
A WAY OF VIEWING WORLD — AGENTS

- OOP uses an approach of treating a real-world agent as an object.
- Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.
- An object-oriented program can be characterized as data controlling access to code by switching the controlling entity to data.

MESSAGE PASSING AMONG OBJECTS IN A PROGRAM SHELL



A WAY OF VIEWING WORLD WITH OOP



RESPONSIBILITY

- primary motivation is the need for a platform-independent (that is, architecture- neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
- Objects with clear responsibilities
- Each class should have a clear responsibility.
- If you can't state the purpose of a class in a single, clear sentence, then perhaps your class structure needs some thought.

MESSAGES

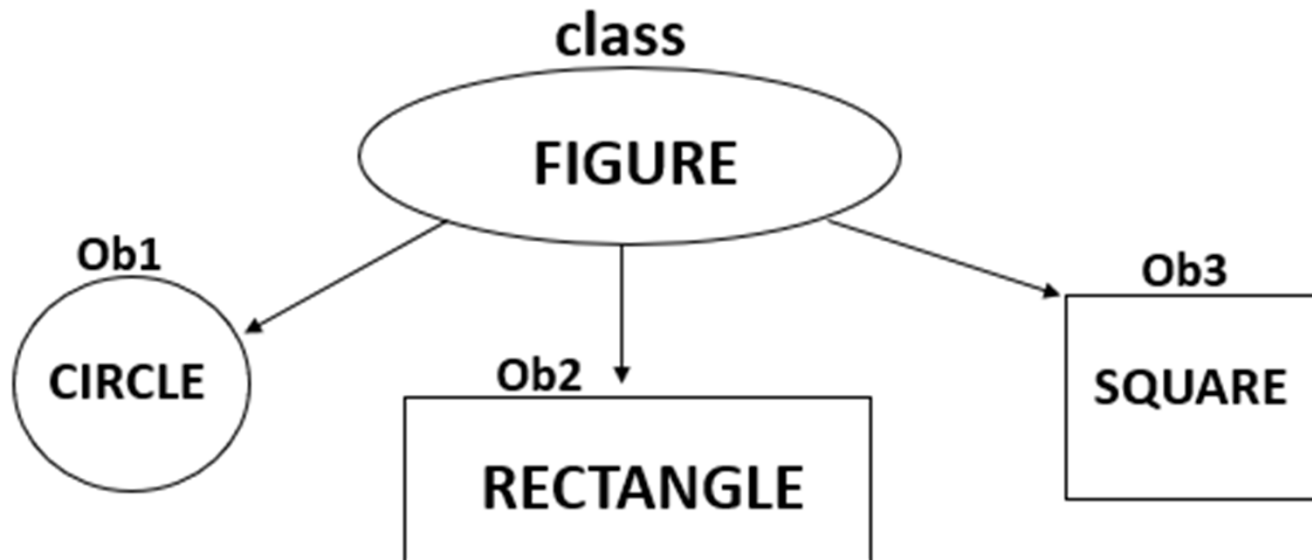
- We all like to use programs that let us know what's going on. Programs that keep us informed often do so by displaying status and error messages.
- These messages need to be translated so they can be understood by end users around the world.
- The Section discusses translatable text messages. Usually, you're done after you move a message String into a ResourceBundle.
- If you've embedded variable data in a message, you'll have to take some extra steps to prepare it for translation.

METHODS

- A method is a group of instructions that is given a name and can be called up at any point in a program simply by quoting that name.
- Drawing a Triangle require draw of three straight lines. This instruction three times to draw a simple triangle.
- We can define a method to call this instruction three times and draw the triangle(i.e. create a method drawLine() to draw lines and this method is called repeatedly to achieve the needed task)
- The idea of methods appears in all programming languages, although sometimes it goes under the name functions and sometimes under the name procedures.
- The name methods is a throw-back to the language C++, from which Java was developed.
- In C++, there is an object called a class which can contain methods. However, everything in Java is enclosed within a class .so the functions within it are called methods

CLASSES

- Class is blue print or an idea of an Object
- From One class any number of Instances can be created
- It is an encapsulation of attributes and methods



SYNTAX OF CLASS

```
class <ClassName>{  
    attributes/variables;  
    Constructors();  
    methods();  
}
```

INSTANCE

- Instance is an Object of a class which is an entity with its own attribute values and methods.
- Creating an Instance

```
ClassName refVariable;
```

```
refVariable = new Constructor();
```

or

```
ClassName refVariable = new Constructor();
```

JAVA CLASS HIERARCHY

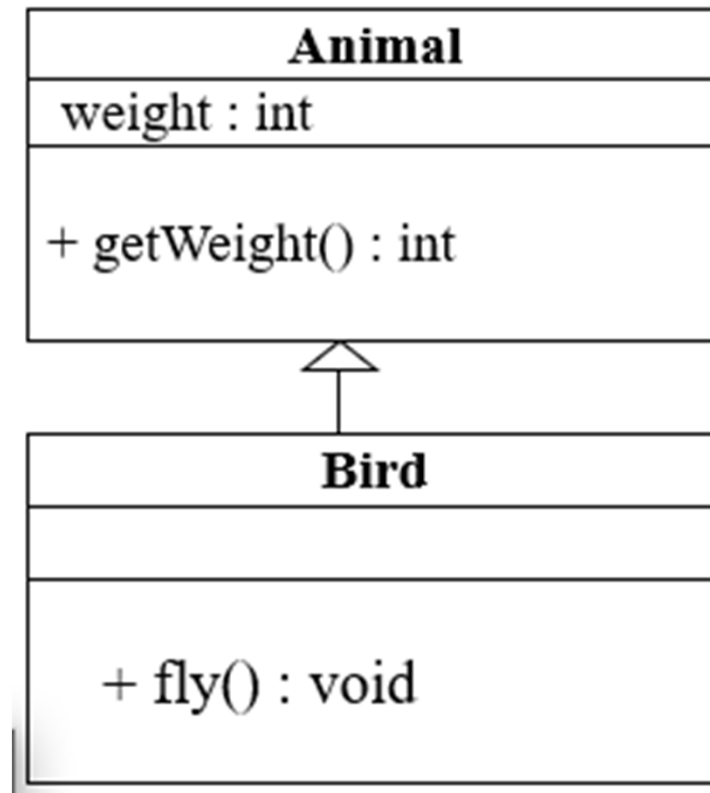
- In Java, class “Object” is the base class to all other classes
 - If we do not explicitly say extends in a new class definition, it implicitly extends Object
 - The tree of classes that extend from Object and all of its subclasses are is called the class hierarchy
 - All classes eventually lead back up to Object
 - This will enable consistent access of objects of different classes.

INHERITANCE

- Methods allows to reuse a sequence of statements
- Inheritance allows to reuse classes by deriving a new class from an existing one
- The existing class is called the parent class, or superclass, or base class
- The derived class is called the child class or subclass.
- The child class inherits characteristics of the parent class(i.e the child class inherits the methods and data defined for the parent class

INHERITANCE

- Inheritance relationships are often shown graphically in a class diagram, with the arrow pointing to the parent class



METHOD BINDING

- Objects are used to call methods.
- Method Binding is an object that can be used to call an arbitrary public method, on an instance that is acquired by evaluating the leading portion of a method binding expression via a value binding.
- It is legal for a class to have two or more methods with the same name.
- Java has to be able to uniquely associate the invocation of a method with its definition, relying on the number and types of arguments.
- Therefore the same-named methods must be distinguished:
 - 1) by the number of arguments, or
 - 2) by the types of arguments
- Overloading and inheritance are two ways to implement polymorphism.

METHOD OVERRIDING.

- There may be some occasions when we want an object to respond to the same method but have different behaviour when that method is called.
- That means, we should override the method defined in the superclass. This is possible by defining a method in a sub class that has the same name, same arguments and same return type as a method in the superclass.
- Then when that method is called, the method defined in the sub class is invoked and executed instead of the one in the superclass. This is known as overriding.

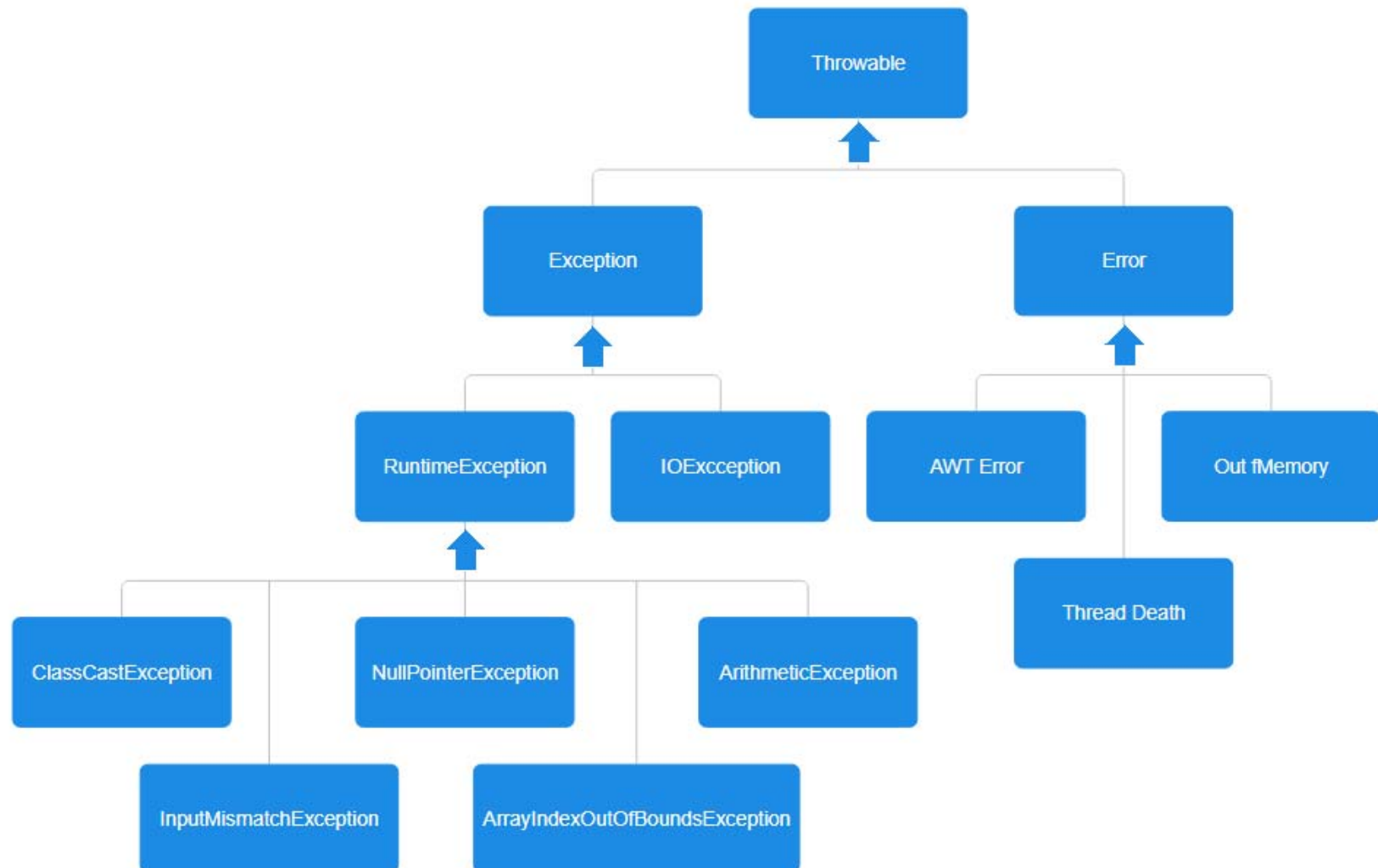
EXCEPTIONS IN JAVA

- Exception is an abnormal condition that arises in the code sequence.
- Exceptions occur during compile time or run time.
- “throwable” is the super class in exception hierarchy.
- Compile time errors occurs due to incorrect syntax.
- Run-time errors happen when
 - User enters incorrect input
 - Resource is not available (ex. file)
 - Logic error (bug) that was not fixed

EXCEPTION CLASSES

- In Java, exceptions are objects. When you throw an exception, you throw an object. You can't throw just any object as an exception, however -- only those objects whose classes descend from Throwable.
- Throwable serves as the base class for an entire family of classes, declared in java.lang, that your program can instantiate and throw.
- Throwable has two direct subclasses, Exception and Error.
- Exceptions are thrown to signal abnormal conditions that can often be handled by some catcher, though it's possible they may not be caught and therefore could result in a dead thread.
- Errors are usually thrown for more serious problems, such as OutOfMemoryError, that may not be so easy to handle. In general, code you write should throw only exceptions, not errors.
- Errors are usually thrown by the methods of the Java API, or by the Java virtual machine itself.

EXCEPTION CLASS HIERARCHY



HISTORY OF JAVA

- Created By: James Gosling and team at Sun Microsystems
- First Released: 1995
- Original Name: Oak (renamed to Java in 1995)
- Purpose: Initially developed for embedded systems and digital devices (like set-top boxes)
- Key Milestones:
 - 1991 Project started as "Oak"
 - 1995 Renamed to Java, first public release (Java 1.0)
 - 1998 Java 2 (J2SE) introduced
 - 2006 Java became open source (OpenJDK)
 - 2010 Oracle acquired Sun Microsystems
 - 2017 Switch to time-based release cycle (every 6 months)

JAVA BUZZWORDS (PRINCIPLES)

- Java was designed with certain principles (buzzwords) that made it powerful and platform-independent.

Buzzword	Meaning
Simple	Easy to learn; similar to C/C++ but with less complexity
Object-Oriented	Everything is treated as an object
Robust	Strong memory management, exception handling, and type checking
Secure	Built-in security features like bytecode verification and sandboxing
Platform-Independent	"Write once, run anywhere" via Java Virtual Machine (JVM)
Portable	Java programs can run on different architectures without modification
High Performance	Just-In-Time (JIT) compiler helps speed up performance
Multithreaded	Supports multiple threads for concurrent programming
Distributed	Built-in support for networking and remote method invocation
Dynamic	Supports dynamic linking and runtime class loading

JAVA KEYWORDS

- Java has a fixed set of reserved words (keywords) that cannot be used as identifiers (variable/class names).
- List of All 67 Java Keywords (as of Java 17):

<code>abstract</code>	<code>continue</code>	<code>for</code>	<code>new</code>	<code>switch</code>
<code>assert</code>	<code>default</code>	<code>goto*</code>	<code>package</code>	<code>synchronized</code>
<code>boolean</code>	<code>do</code>	<code>if</code>	<code>private</code>	<code>this</code>
<code>break</code>	<code>double</code>	<code>implements</code>	<code>protected</code>	<code>throw</code>
<code>byte</code>	<code>else</code>	<code>import</code>	<code>public</code>	<code>throws</code>
<code>case</code>	<code>enum</code>	<code>instanceof</code>	<code>return</code>	<code>transient</code>
<code>catch</code>	<code>extends</code>	<code>int</code>	<code>short</code>	<code>try</code>
<code>char</code>	<code>final</code>	<code>interface</code>	<code>static</code>	<code>void</code>
<code>class</code>	<code>finally</code>	<code>long</code>	<code>strictfp</code>	<code>volatile</code>
<code>const*</code>	<code>float</code>	<code>native</code>	<code>super</code>	<code>while</code>

'goto' and 'const' are reserved but not used in Java.

DATA TYPES

- Java defines eight simple types:
 - 1) byte – 8-bit integer type
 - 2) short – 16-bit integer type
 - 3) int – 32-bit integer type
 - 4) long – 64-bit integer type
 - 5) float – 32-bit floating-point type
 - 6) double – 64-bit floating-point type
 - 7) char – symbols in a character set
 - 8) boolean – logical values true and false

DATA TYPES

- **byte**: 8-bit integer type.

Range: -128 to 127.

Example: `byte b = -15;`

Usage: particularly when working with data streams.

- **short**: 16-bit integer type.

Range: -32768 to 32767.

Example: `short c = 1000;`

Usage: probably the least used simple type.

DATA TYPES

- **int**: 32-bit integer type.

Range: -2147483648 to 2147483647.

Example: `int b = -50000;`

Usage:

- 1) Most common integer type.
- 2) Typically used to control loops and to index arrays.
- 3) Expressions involving the byte, short and int values are promoted to int before calculation.

DATA TYPES

- **long:** 64-bit integer type.

Range: -9223372036854775808 to
9223372036854775807.

Example: `long l = 1000000000000000000;`

Usage: 1) useful when the `int` type is not large enough to hold the desired value

- **Float:** 32-bit floating-point number.

Range: $1.4e-045$ to $3.4e+038$.

Example: `float f = 1.5;`

Usage:

- 1) The fractional part is needed
- 2) A large degree of precision is not required

DATA TYPES

- **double:** 64-bit floating-point number.
Range: $4.9e-324$ to $1.8e+308$.
Example: `double pi = 3.1416;`
Usage:
 - 1) accuracy over many iterative calculations
 - 2) manipulation of large-valued numbers

DATA TYPES

char: 16-bit data type used to store characters.

Range: 0 to 65536.

Example: `char c = 'a';`

Usage:

- 1) Represents both ASCII and Unicode character sets; Unicode defines a character set with characters found in (almost) all human languages.
- 2) Not the same as in C/C++ where char is 8-bit and represents ASCII only.

DATA TYPES

- **boolean:** Two-valued type of logical values.
Range: values true and false.
Example: `boolean b = (1<2);`
Usage:
 - 1) returned by relational operators, such as `1<2`
 - 2) required by branching expressions such as `if` or `for`

VARIABLES

- declaration – how to assign a type to a variable
- initialization – how to give an initial value to a variable
- scope – how the variable is visible to other parts of the program
- lifetime – how the variable is created, used and destroyed
- type conversion – how Java handles automatic type conversion
- type casting – how the type of a variable can be narrowed down
- type promotion – how the type of a variable can be expanded

VARIABLES

- Java uses variables to store data.
- To allocate memory space for a variable JVM requires:
 - 1) to specify the data type of the variable
 - 2) to associate an identifier with the variable
 - 3) optionally, the variable may be assigned an initial value
- All done as part of variable declaration.

BASIC VARIABLE DECLARATION

- datatype identifier [=value];
- datatype must be
 - A simple datatype
 - User defined datatype (class type)
- Identifier is a recognizable name conform to identifier rules
- Value is an optional initial value.

VARIABLE DECLARATION

- We can declare several variables at the same time:

`type identifier [=value][, identifier [=value] ...];`

Examples:

`int a, b, c;`

`int d = 3, e, f = 5;`

`byte g = 22;`

`double pi = 3.14159;`

`char ch = 'x';`

VARIABLE SCOPE

- Scope determines the visibility of program elements with respect to other program elements.
- In Java, scope is defined separately for classes and methods:
 - 1) variables defined by a class have a global scope
 - 2) variables defined by a method have a local scope

A scope is defined by a block:

```
{    ...    }
```

A variable declared inside the scope is not visible outside:

```
{    int n;    }
```

```
n = 1; // this is illegal
```

VARIABLE LIFETIME

- Variables are created when their scope is entered by control flow and destroyed when their scope is left:
- A variable declared in a method will not hold its value between different invocations of this method.
- A variable declared in a block loses its value when the block is left.
- Initialized in a block, a variable will be re-initialized with every re-entry. Variables lifetime is confined to its scope!

ARRAYS

- An array is a group of liked-typed variables referred to by a common
- name, with individual variables accessed by their index.
- Arrays are:
 1. declared
 2. created
 3. initialized
 4. used
- Also, arrays can have one or several dimensions.

ARRAY DECLARATION

- Array declaration involves:
 - 1) declaring an array identifier
 - 2) declaring the number of dimensions
 - 3) declaring the data type of the array elements
- Two styles of array declaration:

```
type array-variable[];
```

or

```
type [] array-variable;
```

ARRAY CREATION

- After declaration, no array actually exists.
- In order to create an array, we use the new operator:

```
type array-variable[];  
array-variable = new type[size];
```
- This creates a new array to hold size elements of type type, which reference will be kept in the variable array-variable.

ARRAY INDEXING

- Later we can refer to the elements of this array through their indexes:
- `array-variable[index]`
- The array index always starts with zero!
- The Java run-time system makes sure that all array indexes are in the correct range, otherwise raises a run-time error.

ARRAY INITIALIZATION

- Arrays can be initialized when they are declared:

```
int monthDays[] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

- Note:
 1. there is no need to use the new operator
 2. the array is created large enough to hold all specified elements

MULTIDIMENSIONAL ARRAYS

- Multidimensional arrays are arrays of arrays:
 - 1) declaration: `int array[][];`
 - 2) creation: `int array = new int[2][3];`
 - 3) initialization
`int array[][] = { {1, 2, 3}, {4, 5, 6} };`

OPERATORS TYPES

- Java operators are used to build value expressions.
- Java provides a rich set of operators:
 - 1) assignment
 - 2) arithmetic
 - 3) relational
 - 4) logical
 - 5) bitwise

ARITHMETIC ASSIGNMENTS

<code>+=</code>	<code>v += expr;</code>	<code>v = v + expr ;</code>
<code>-=</code>	<code>v -=expr;</code>	<code>v = v - expr ;</code>
<code>*=</code>	<code>v *= expr;</code>	<code>v = v * expr ;</code>
<code>/=</code>	<code>v /= expr;</code>	<code>v = v / expr ;</code>
<code>%=</code>	<code>v %= expr;</code>	<code>v = v % expr ;</code>

BASIC ARITHMETIC OPERATORS

+	$\text{op1} + \text{op2}$	ADD
-	$\text{op1} - \text{op2}$	SUBTRACT
*	$\text{op1} * \text{op2}$	MULTIPLY
/	$\text{op1} / \text{op2}$	DIVISION
%	$\text{op1} \% \text{op2}$	REMAINDER

RELATIONAL OPERATOR

==	Equals to	Apply to any type
!=	Not equals to	Apply to any type
>	Greater than	Apply to numerical type
<	Less than	Apply to numerical type
>=	Greater than or equal	Apply to numerical type
<=	Less than or equal	Apply to numerical type

LOGICAL OPERATORS

&	op1 & op2	Logical AND
	op1 op2	Logical OR
&&	op1 && op2	Short-circuit AND
	op1 op2	Short-circuit OR
!	! op	Logical NOT
^	op1 ^ op2	Logical XOR

BIT WISE OPERATORS

~	~op	Inverts all bits
&	op1 & op2	Produces 1 bit if both operands are 1
	op1 op2	Produces 1 bit if either operand is 1
^	op1 ^ op2	Produces 1 bit if exactly one operand is 1
>>	op1 >> op2	Shifts all bits in op1 right by the value of op2
<<	op1 << op2	Shifts all bits in op1 left by the value of op2

EXPRESSIONS

- An *expression* is a construct made up of variables, operators, and method invocations, which are constructed according to the syntax of the language, that evaluates to a single value.

- Examples of expressions are in bold below:

```
int number = 0;
```

```
anArray[0] = 100;
```

```
System.out.println ("Element 1 at index 0: " + anArray[0]);
```

```
int result = 1 + 2; // result is now 3
```

```
if(value1 == value2)
```

```
    System.out.println("value1 == value2");
```

EXPRESSIONS

- The data type of the value returned by an expression depends on the elements used in the expression.
- The expression `number = 0` returns an `int` because the assignment operator returns a value of the same data type as its left-hand operand; in this case, `number` is an `int`.
- As you can see from the other expressions, an expression can return other types of values as well, such as `boolean` or `String`.
- The Java programming language allows you to construct compound expressions from various smaller expressions as long as the data type required by one part of the expression matches the data type of the other.
- Here's an example of a compound expression: $1 * 2 * 3$

CONTROL STATEMENTS

- Java control statements cause the flow of execution to advance and branch based on the changes to the state of the program.
- Control statements are divided into three groups:
- 1) selection statements allow the program to choose different parts of the execution based on the outcome of an expression
- 2) iteration statements enable program execution to repeat one or more statements
- 3) jump statements enable your program to execute in a non-linear fashion

SELECTION STATEMENTS

- Java selection statements allow to control the flow of program's execution based upon conditions known only during run-time.
- Java provides four selection statements:
 - 1) if
 - 2) if-else
 - 3) if-else-if
 - 4) switch

ITERATION STATEMENTS

- Java iteration statements enable repeated execution of part of a program until a certain termination condition becomes true.
- Java provides three iteration statements:
 - 1) while
 - 2) do-while
 - 3) for

JUMP STATEMENTS

- Java jump statements enable transfer of control to other parts of program.
- Java provides three jump statements:
 - 1) break
 - 2) continue
 - 3) return
- In addition, Java supports exception handling that can also alter the control flow of a program.

TYPE CONVERSION

- Size Direction of Data Type
 - Widening Type Conversion (Casting down)
 - Smaller Data Type → Larger Data Type
 - Narrowing Type Conversion (Casting up)
 - Larger Data Type → Smaller Data Type
- Conversion done in two ways
 - Implicit type conversion
 - Carried out by compiler automatically
 - Explicit type conversion
 - Carried out by programmer using casting

TYPE CONVERSION

- Widening Type Conversion
 - Implicit conversion by the compiler automatically

byte -> short, int, long, float, double
short -> int, long, float, double
char -> int, long, float, double
int -> long, float, double
long -> float, double
float -> double

TYPE CONVERSION

- **Narrowing Type Conversion**
 - The programmer should describe the conversion explicitly

byte -> char
short -> byte, char
char -> byte, short
int -> byte, short, char
long -> byte, short, char, int
float -> byte, short, char, int, long
double -> byte, short, char, int, long, float

TYPE CONVERSION

- byte and short are always promoted to int
- if one operand is long, the whole expression is promoted to long
- if one operand is float, the entire expression is promoted to float
- if any operand is double, the result is double

TYPE CASTING

- General form: (targetType) value
- Examples:
 1. integer value will be reduced module bytes range:
 int i;
 byte b = (byte) i;
 2. floating-point value will be truncated to integer value:
 float f;
 int i = (int) f;

SIMPLE JAVA PROGRAM

- A class to display a simple message:

```
class MyProgram{  
    public static void main(String[] args){  
        System.out.println("First Java program.");  
    }  
}
```

WHAT IS AN OBJECT?

- Real world objects are things that have:

- 1) state

- 2) behavior

Example: your dog:

- **state** – name, color, breed, sits?, barks?, wags tail?, runs?
- **behavior** – sitting, barking, wagging tail, running
- A software object is a bundle of variables (state) and methods (operations).

WHAT IS A CLASS?

- A class is a blueprint that defines the variables and methods common to all objects of a certain kind.
- Example: 'your dog' is a object of the class Dog.
- An object holds values for the variables defines in the class.
- An object is called an instance of the Class

OBJECT CREATION

- A variable is declared to refer to the objects of type/class String:

String s;

- The value of s is null; it does not yet refer to any object.
- A new String object is created in memory with initial "abc" value:

String s = new String("abc");

- Now s contains the address of this new object.

OBJECT DESTRUCTION

- A program accumulates memory through its execution.
- Two mechanism to free memory that is no longer need by the program:
 - 1) manual – done in C/C++
 - 2) automatic – done in Java
- In Java, when an object is no longer accessible through any variable, it is eventually removed from the memory by the garbage collector.
- Garbage collector is parts of the Java Run-Time Environment.

CLASS

- A basis for the Java language.
- Each concept we wish to describe in Java must be included inside a class.
- A class defines a new data type, whose values are objects:
- A class is a template for objects
- An object is an instance of a class

CLASS DEFINITION

- A class contains a name, several variable declarations (instance variables) and several method declarations. All are called members of the class.
- General form of a class:

```
class classname {  
    type instance-variable-1;  
    ... ..  
    type instance-variable-n;  
    type method-name-1(parameter-list) { ... }  
    type method-name-2(parameter-list) { ... }  
    ... ..  
    type method-name-m(parameter-list) { ... }  
}
```

EXAMPLE: CLASS USAGE

```
class Box {  
    double width;  
    double height;  
    double depth;  
}  
class BoxDemo {  
    public static void main(String args[]) {  
        Box mybox = new Box();  
        double vol;  
        mybox.width = 10;  
        mybox.height = 20;  
        mybox.depth = 15;  
        vol = mybox.width * mybox.height * mybox.depth;  
        System.out.println ("Volume is " + vol);  
    }  
}
```

CONSTRUCTOR

- A constructor initializes the instance variables of an object.
- It is called immediately after the object is created but before the new operator completes.
 - 1) it is syntactically similar to a method:
 - 2) it has the same name as the name of its class
 - 3) it is written without return type; the default return type of a class
- constructor is the same class
- When the class has no constructor, the default constructor automatically initializes all its instance variables with zero.

EXAMPLE: CONSTRUCTOR

```
class Box {  
    double width;  
    double height;  
    double depth;  
    Box() {  
        System.out.println("Constructing Box");  
        width = 10; height = 10; depth = 10;  
    }  
    double volume() {  
        return width * height * depth;  
    }  
}
```

PARAMETERIZED CONSTRUCTOR

```
class Box {  
    double width, height, depth;  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}  
  
class Box {  
    double width;  
    double height;  
    double depth;  
    Box(double w, double h, double d) {  
        width = w; height = h; depth = d;  
    }  
    double volume(){  
        return width * height * depth;  
    }  
}
```

METHODS

- General form of a method definition:

```
type name(parameter-list) {  
    ... return value;  
    ...  
}
```

- Components:

- 1) type - type of values returned by the method. If a method does not return any value, its return type must be void.
- 2) name is the name of the method
- 3) parameter-list is a sequence of type-identifier lists separated by commas
- 4) return value indicates what value is returned by the method.

EXAMPLE: METHOD

- Classes declare methods to hide their internal data structures, as well as for their own internal use: Within a class, we can refer directly to its member variables:

```
class Box {  
    double width, height, depth;  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}
```

PARAMETERIZED METHOD

- Parameters increase generality and applicability of a method:

- 1) method without parameters

```
int square() { return 10*10; }
```

- 2) method with parameters

```
int square(int i) { return i*i; }
```

- Parameter: a variable receiving value at the time the method is invoked.
- Argument: a value passed to the method when it is invoked.

ACCESS CONTROL: DATA HIDING AND ENCAPSULATION

- Java provides control over the *visibility* of variables and methods.
- *Encapsulation*, safely sealing data within the *capsule* of the class Prevents programmers from relying on details of class implementation, so you can update without worry
- Helps in protecting against accidental or wrong usage.
- Keeps code elegant and clean (easier to maintain)

ACCESS MODIFIERS: PUBLIC, PRIVATE, PROTECTED

- *Public*: keyword applied to a class, makes it available/visible everywhere. Applied to a method or variable, completely visible.
- Default (No visibility modifier is specified): it behaves like public in its package and private in other packages.
- *Default Public* keyword applied to a class, makes it available/visible everywhere. Applied to a method or variable, completely visible.

ACCESS MODIFIERS: PUBLIC, PRIVATE, PROTECTED

- *Private* fields or methods for a class only visible within that class. Private members are *not* visible within subclasses, and are *not* inherited.
- *Protected* members of a class are visible within the class, subclasses and *also* within all classes that are in the same package as that class.

VISIBILITY

```
public class Circle {  
    private double x,y,r;  
  
    // Constructor  
    public Circle (double x, double y, double r) {  
        this.x = x;  
        this.y = y;  
        this.r = r;  
    }  
  
    //Methods to return circumference and area  
    public double circumference() { return 2*3.14*r;}  
    public double area() { return 3.14 * r * r; }  
}
```

KEYWORD THIS

- Can be used by any object to refer to itself in any class method
- Typically used to
 - Avoid variable name collisions
 - Pass the receiver as an argument
 - Chain constructors

KEYWORD THIS

- Keyword this allows a method to refer to the object that invoked it.
- It can be used inside any method to refer to the current object:

```
Box(double width, double height, double depth) {  
    this.width = width;  
    this.height = height;  
    this.depth = depth;  
}
```

GARBAGE COLLECTION

- Garbage collection is a mechanism to remove objects from memory when they are no longer needed.
- Garbage collection is carried out by the garbage collector:
 - 1) The garbage collector keeps track of how many references an object has.
 - 2) It removes an object from memory when it has no longer any references.
 - 3) Thereafter, the memory occupied by the object can be allocated again.
 - 4) The garbage collector invokes the finalize method.

FINALIZE() METHOD

- A constructor helps to initialize an object just after it has been created.
- In contrast, the finalize method is invoked just before the object is destroyed:
- 1) implemented inside a class as:

```
protected void finalize() { ... }
```
- 2) implemented when the usual way of removing objects from memory is insufficient, and some special actions has to be carried out

METHOD OVERLOADING

- It is legal for a class to have two or more methods with the same name.
- However, Java has to be able to uniquely associate the invocation of a method with its definition relying on the number and types of arguments.
- Therefore the same-named methods must be distinguished:
 - 1) by the number of arguments, or
 - 2) by the types of arguments
- Overloading and inheritance are two ways to implement polymorphism.

EXAMPLE: OVERLOADING

```
class OverloadDemo {  
    void test() {  
        System.out.println("No parameters");  
    }  
    void test(int a) {  
        System.out.println("a: " + a);  
    }  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " " + b);  
    }  
    double test(double a) {  
        System.out.println("double a: " + a); return a*a;  
    }  
}
```

CONSTRUCTOR OVERLOADING

```
class Box {  
    double width, height, depth;  
    Box(double w, double h, double d) {  
        width = w; height = h; depth = d;  
    }  
    Box() {  
        width = -1; height = -1; depth = -1;  
    }  
    Box(double len) {  
        width = height = depth = len;  
    }  
    double volume() {  
        return width * height * depth;  
    }  
}
```

PARAMETER PASSING

- Two types of variables:
 - 1) simple types
 - 2) class types
- Two corresponding ways of how the arguments are passed to methods:
 - 1) by value a method receives a copy of the original value; parameters of simple types
 - 2) by reference a method receives the memory address of the original value, not the value itself; parameters of class types

CALL BY VALUE

```
class CallByValue {  
    public static void main(String args[]) {  
        Test ob = new Test();  
        int a = 15, b = 20;  
        System.out.print("a and b before call: ");  
        System.out.println(a + " " + b);  
        ob.meth(a, b);  
        System.out.print("a and b after call: ");  
        System.out.println(a + " " + b);  
    }  
}
```

CALL BY REFERENCE

- As the parameter hold the same address as the argument, changes to the object inside the method do affect the object used by the argument:

```
class CallByRef {  
    public static void main(String args[]) {  
        Test ob = new Test(15, 20);  
        System.out.print("ob.a and ob.b before call: ");  
        System.out.println(ob.a + " " + ob.b);  
        ob.meth(ob);  
        System.out.print("ob.a and ob.b after call: ");  
        System.out.println(ob.a + " " + ob.b);  
    }  
}
```

RECURSION

- A recursive method is a method that calls itself:
 - 1) all method parameters and local variables are allocated on the stack
 - 2) arguments are prepared in the corresponding parameter positions
 - 3) the method code is executed for the new arguments
 - 4) upon return, all parameters and variables are removed from the stack
 - 5) the execution continues immediately after the invocation point

EXAMPLE: RECURSION

```
class Factorial {  
    int fact(int n) {  
        if (n==1) return 1;  
        return fact(n-1) * n;  
    }  
}  
  
class Recursion {  
    public static void main(String args[]) {  
        Factorial f = new Factorial();  
        System.out.print("Factorial of 5 is ");  
        System.out.println(f.fact(5));  
    }  
}
```

STRING HANDLING

- **String** is probably the most commonly used class in Java's class library. The obvious reason for this is that strings are a very important part of programming.
- The first thing to understand about strings is that every string you create is actually an object of type **String**. Even string constants are actually **String** objects.
- For example, in the statement
`System.out.println("This is a String, too");`
the string "This is a String, too" is a **String** constant

STRING HANDLING

- Java defines one operator for **String** objects: **+**.
- It is used to concatenate two strings. For example, this statement
- `String myString = "I" + " like " + "Java.";`
results in **myString** containing
"I like Java."

STRING HANDLING

- The **String** class contains several methods that you can use. Here are a few. You can
- test two strings for equality by using **equals()**. You can obtain the length of a string by calling the **length()** method. You can obtain the character at a specified index within a string by calling **charAt()**.
- The general forms of these three methods are shown here:

STRING HANDLING

```
// Demonstrating some String methods.
class StringDemo2 {
    public static void main(String args[]) {
        String strOb1 = "First String";
        String strOb2 = "Second String";
        String strOb3 = strOb1;
        System.out.println("Length of strOb1: " + strOb1.length());
        System.out.println ("Char at index 3 in strOb1: " + strOb1.charAt(3));
        if(strOb1.equals(strOb2))    System.out.println("strOb1 == strOb2");
        else    System.out.println("strOb1 != strOb2");
        if(strOb1.equals(strOb3))    System.out.println("strOb1 == strOb3");
        else    System.out.println("strOb1 != strOb3");
    }
}
```

This program generates the following output:

```
Length of strOb1: 12
Char at index 3 in strOb1: s
strOb1 != strOb2
strOb1 == strOb3
```